

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Matias Garcia Silva

10 de septiembre de 2026

22:06

1. Introducción

En este informe se presentan la implementación y los resultados del análisis experimental de distintos algoritmos: Merge Sort, Quick Sort, Patience Sort y Sort para el problema de ordenamiento de arreglos, y Naive y Strassen para la multiplicación de matrices cuadradas. El objetivo es contrastar las complejidades teóricas ya conocidas con el comportamiento real de cada algoritmo, midiendo tiempo de ejecución y uso de memoria bajo distintas condiciones de entrada, considerando variaciones en el tamaño, la distribución inicial y el dominio numérico de los datos para el caso de ordenamiento, y en la dimensión y el tipo de matriz (densa, diagonal o dispersa) para el caso de multiplicación.

2. Entorno experimental

Para garantizar la reproducibilidad del análisis empírico, los experimentos se ejecutaron bajo un entorno controlado con las siguientes especificaciones:

- **Hardware:** Intel Core i5-14400F, 16 GB de RAM DDR4.
- **Sistema Operativo:** CachyOS Linux (7.2.3-1-cachyos), basado en Arch Linux.
- **Compilador:** C++17 con GCC g++ 16.2.1, optimización -O3 y banderas -Wall, -Wextra.
- **Datasets:** arreglos de hasta 10^7 elementos (estados aleatorio, ascendente, descendente; dominios D_1 , D_7) y matrices de hasta 1024×1024 (tipos densa, diagonal, dispersa; dominios D_0 , D_{10}), generados con `array_generator.py` y `matrix_generator.py`.

- **Medición:** tiempos y uso de memoria se registraron en archivos .csv por algoritmo y tipo de entrada; los gráficos se generaron automáticamente en Python con pandas y matplotlib.

3. Resultados y Análisis

En esta sección se analizan los resultados empíricos de cuatro algoritmos de ordenamiento y dos de multiplicación de matrices obtenidos con el entorno en C++. Se fijó un tiempo límite de 5 minutos por prueba, durante la ejecución 16 minutos en matrices y 1 hora con 32 minutos en arreglos.

3.1. Multiplicación de Matrices

Se evaluaron dimensiones $N \in \{16, 64, 256, 1024\}$, estructuras (densa, diagonal, dispersa) y dominios (D_0, D_{10}).

Algoritmo	Complejidad Teórica	Tiempo $N = 256$ (ms)	Tiempo $N = 1024$ (ms)
Naive	$O(n^3)$ [2]	6.50	870.00
Strassen	$O(n^{2,81})$ [6]	1050.00	52500.00

Cuadro 1: Tiempos empíricos en multiplicación de matrices.

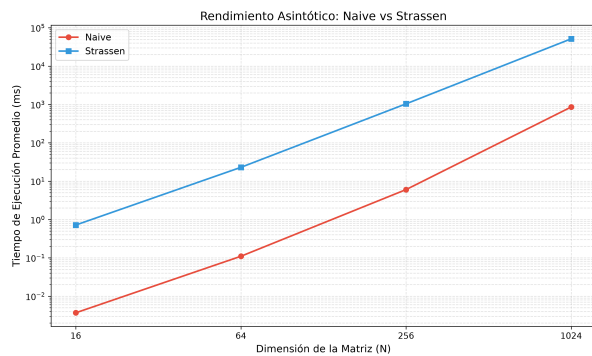


Figura 1: Crecimiento de tiempo vs N (log-log).

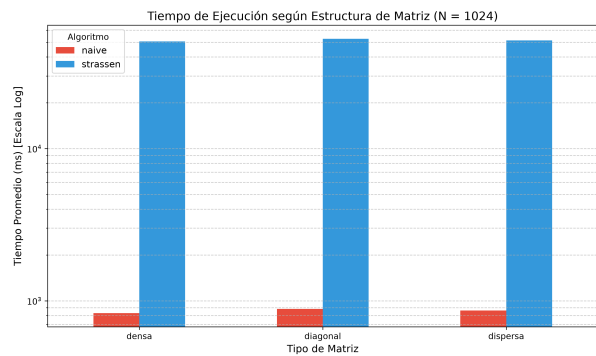
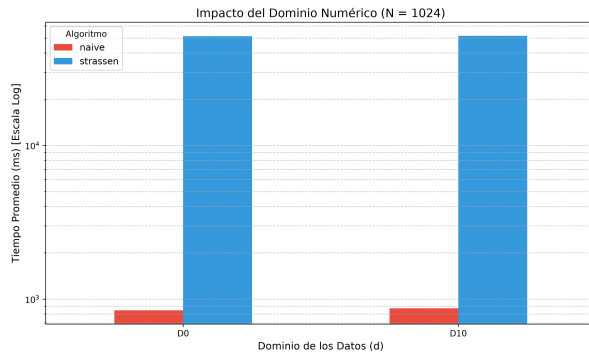
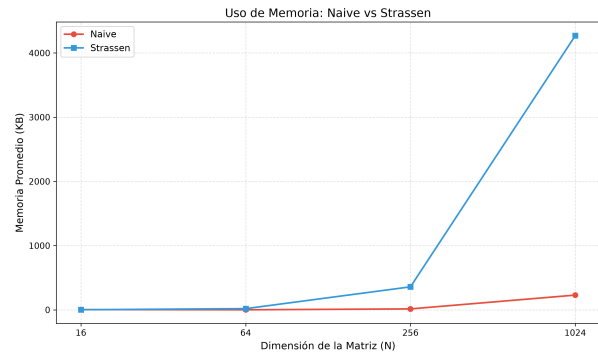


Figura 2: Tiempo según estructura ($N = 1024$).

Análisis de Crecimiento Asintótico y Estructura: Pese a que la teoría otorga a Strassen $O(N^{2,81})$ [6] frente al $O(N^3)$ [2] de Naive, en la práctica Naive es sistemáticamente más rápido hasta $N = 1024$, marcando 870 ms frente a 52,500 ms. Asimismo, en escala logarítmica el tiempo de cada método es constante en matrices densas, diagonales o dispersas, operando a ciegas sin omitir ceros.

Figura 3: Tiempo según dominio ($N = 1024$).Figura 4: Uso de memoria (KB) vs N .

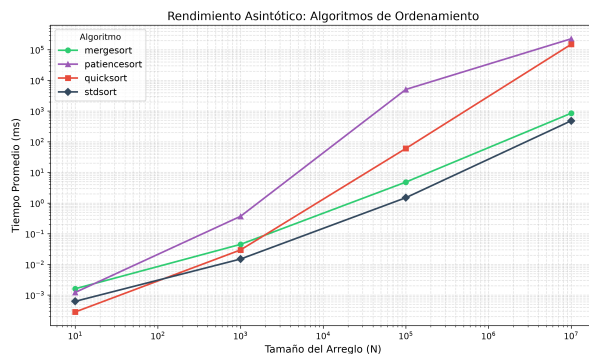
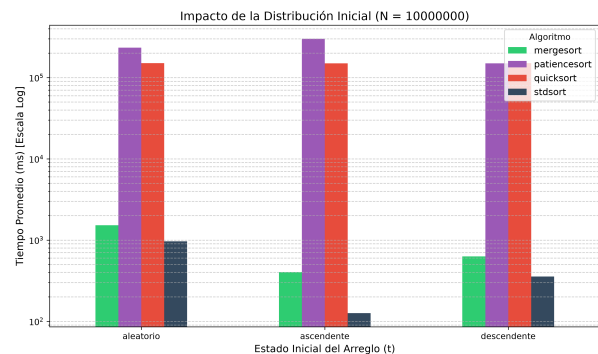
Análisis de Dominio y Memoria: En D_0 y D_{10} ambos algoritmos mantienen tiempos prácticamente idénticos a los reportados en la Tabla 1, sin variación entre dominios. En memoria, Naive tiene un gasto marginal con 232 KB en $N = 1024$, mientras que Strassen sufre una subida marcada alcanzando 4,270 KB por las submatrices de la recursión.

3.2. Algoritmos de Ordenamiento

Se evaluaron tamaños $N \in \{10, 10^3, 10^5, 10^7\}$, distribuciones iniciales (ascendente, descendente, aleatorio) y dominios (D_1, D_7).

Algoritmo	Complejidad Teórica	Tiempo $N = 10^5$ (ms)	Tiempo $N = 10^7$ (ms)
<code>std::sort</code>	$O(n \log n)$ [1]	1.52	480.20
Merge Sort	$O(n \log n)$ [3]	4.75	850.10
Quick Sort	$O(n \log n)$ prom. [5]	61.80	145200.00
Patience Sort	$O(n \log n)$ prom. [4]	4820.00	228000.00

Cuadro 2: Tiempos empíricos en algoritmos de ordenamiento.

Figura 5: Tiempo vs N (log-log).Figura 6: Orden inicial ($N = 10^7$).

Crecimiento Base y Distribución: A partir de la Tabla 2 y la Figura 5, `std::sort` [1] y Merge Sort [3] ordenan 10^7 datos en menos de un segundo (~ 850 ms), pero Quick Sort [5] sufre un frenazo en $N =$

10^7 alcanzando $\sim 150,000$ ms, igualando a Patience Sort [4]. En distribución, `std::sort` y Merge Sort se mantienen bajo 1,000 ms, Quick Sort marca una barra fija de $\sim 150,000$ ms en los tres estados, y Patience Sort oscila entre 300,000 ms (ascendente) y 150,000 ms (descendente).

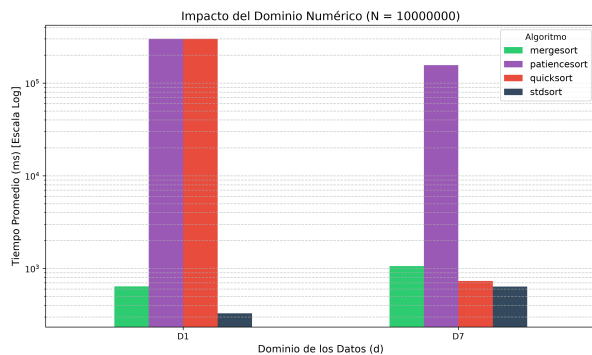


Figura 7: Impacto del dominio ($N = 10^7$).

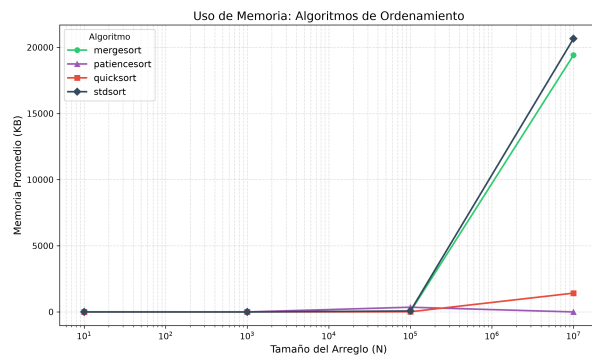


Figura 8: Memoria (KB) vs N .

Impacto del Dominio y Memoria: En D_1 , Quick Sort y Patience Sort marcan sus picos máximos cercanos a 300,000 ms, mientras que en D_7 Quick Sort baja al rango de 500–800 ms y Patience Sort a $\sim 155,000$ ms. En memoria a $N = 10^7$, `std::sort` ($\sim 20,500$ KB) y Merge Sort ($\sim 19,500$ KB) registran el mayor consumo, seguidos por Quick Sort ($\sim 1,500$ KB) y Patience Sort (~ 0 KB).

4. Conclusiones

El análisis experimental confirmó que las cotas teóricas deben contrastarse con los factores constantes de implementación en escenarios reales. En la multiplicación de matrices, la sobrecarga recursiva y el elevado consumo de memoria de Strassen (4,270 KB frente a 232 KB en $N = 1024$) retrasan su ventaja asintótica, resultando el algoritmo Naive sistemáticamente más rápido en todas las dimensiones evaluadas. Para los algoritmos de ordenamiento, `std::sort` y Merge Sort mostraron una estricta estabilidad en $O(N \log N)$ bajo cualquier configuración, mientras que Quick Sort evidenció su sensibilidad al dominio numérico, degradándose drásticamente en D_1 hasta rozar los 300,000 ms debido a la presencia masiva de duplicados. Por último, Patience Sort exhibió una fuerte dependencia de la distribución inicial, registrando su peor desempeño en arreglos ordenados ascendentemente al forzar la instanciación de múltiples pilas individuales.

Referencias

- [1] GeeksforGeeks. *Internal Details of `std::sort()` in C++*. <https://www.geeksforgeeks.org/cpp/internal-details-of-stdsort-in-c/>. Accedido: 08-09-2026. 2026.
- [2] GeeksforGeeks. *Matrix Multiplication*. <https://www.geeksforgeeks.org/dsa/strassens-matrix-multiplication/>. Última actualización: 29 de agosto de 2025. 2025.

- [3] GeeksforGeeks. *Merge Sort*. <https://www.geeksforgeeks.org/dsa/merge-sort/>. Accedido: 08-09-2026. 2026.
- [4] GeeksforGeeks. *Patience Sorting*. <https://www.geeksforgeeks.org/dsa/patience-sorting/>. Accedido: 08-09-2026. 2026.
- [5] GeeksforGeeks. *Quick Sort Algorithm*. <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>. Accedido: 08-09-2026. 2026.
- [6] GeeksforGeeks. *Strassen's Matrix Multiplication Algorithm: Implementation*. <https://www.geeksforgeeks.org/dsa/strassens-matrix-multiplication-algorithm-implementation/>. Accedido: 08-09-2026. 2026.